

Práctica 1

Sistemas Operativos

José Miguel Martínez García

1 de Octubre de 2026

Índice

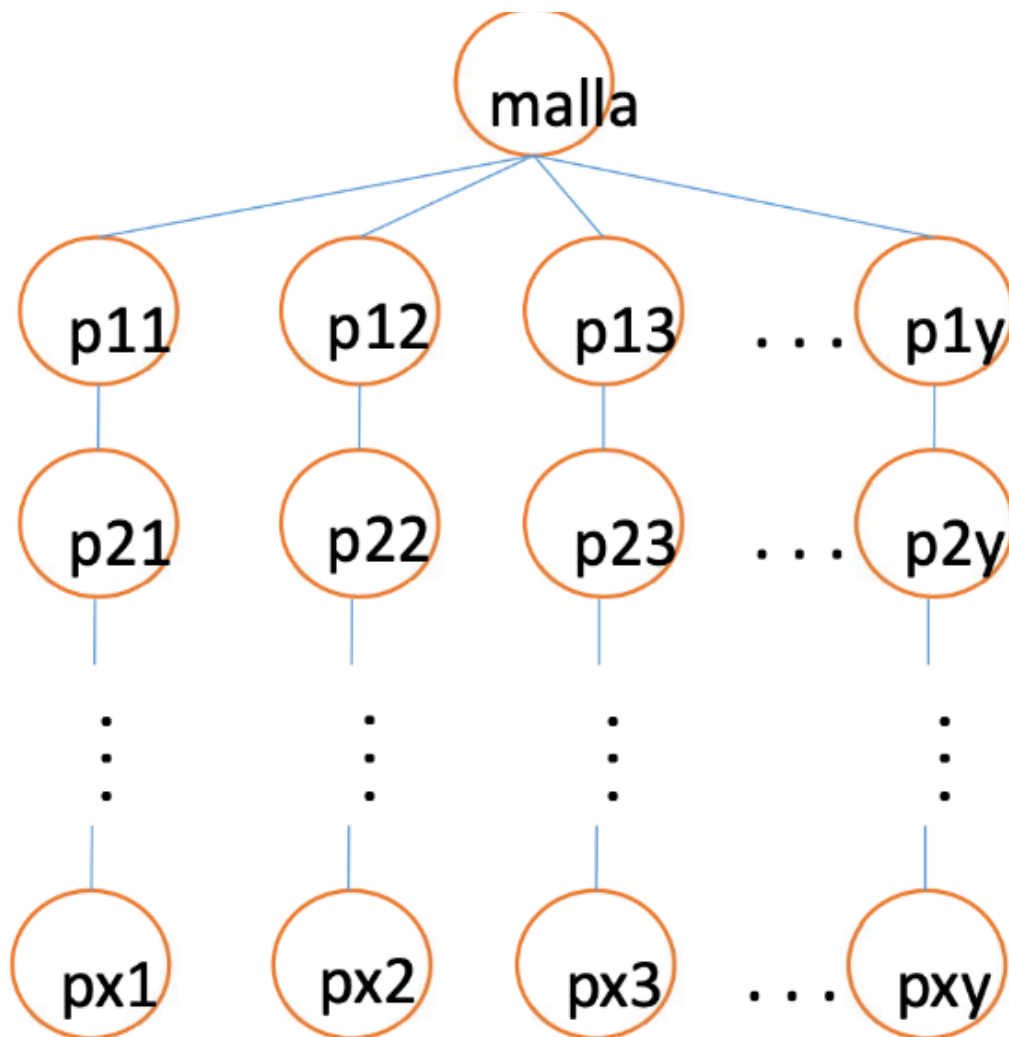
1. Ejercicio 1 A

Gestión Básica de Procesos

1. Enunciado del Ejercicio

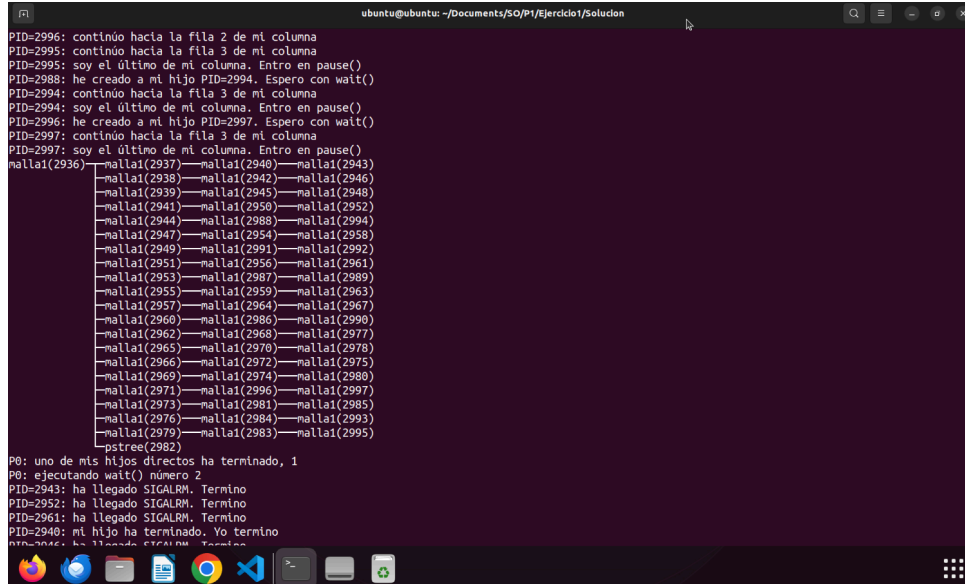
Realiza un programa llamado **mallá.c** que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema “*pstree -c*”.

1.1. Estructura del Árbol de Procesos



1.2.3. Ejemplo 3: Malla 3x20

A continuación se muestra un resumen del árbol de salida de dicho programa:



```
ubuntu@ubuntu: ~/Documents/SO/P1/Ejercicio1/Solucion
PID=2996: continuó hacia la fila 2 de mi columna
PID=2995: continuó hacia la fila 3 de mi columna
PID=2995: soy el último de mi columna. Entro en pause()
PID=2988: he creado a mi hijo PID=2994. Espero con wait()
PID=2994: continuó hacia la fila 3 de mi columna
PID=2994: soy el último de mi columna. Entro en pause()
PID=2996: he creado a mi hijo PID=2997. Espero con wait()
PID=2997: continuó hacia la fila 3 de mi columna
PID=2997: soy el último de mi columna. Entro en pause()
mallai(2936)
  mallai(2937)
  mallai(2940)
  mallai(2943)
  mallai(2938)
  mallai(2942)
  mallai(2946)
  mallai(2939)
  mallai(2945)
  mallai(2948)
  mallai(2941)
  mallai(2950)
  mallai(2952)
  mallai(2944)
  mallai(2988)
  mallai(2994)
  mallai(2947)
  mallai(2954)
  mallai(2958)
  mallai(2949)
  mallai(2991)
  mallai(2992)
  mallai(2951)
  mallai(2956)
  mallai(2961)
  mallai(2953)
  mallai(2987)
  mallai(2989)
  mallai(2955)
  mallai(2959)
  mallai(2963)
  mallai(2957)
  mallai(2964)
  mallai(2967)
  mallai(2960)
  mallai(2986)
  mallai(2990)
  mallai(2962)
  mallai(2969)
  mallai(2977)
  mallai(2965)
  mallai(2970)
  mallai(2970)
  mallai(2966)
  mallai(2972)
  mallai(2975)
  mallai(2969)
  mallai(2974)
  mallai(2980)
  mallai(2971)
  mallai(2996)
  mallai(2997)
  mallai(2973)
  mallai(2981)
  mallai(2985)
  mallai(2976)
  mallai(2984)
  mallai(2993)
  mallai(2979)
  mallai(2983)
  mallai(2995)
  pstree(2982)
P0: uno de mis hijos directos ha terminado, 1
P0: ejecutando wait() número 2
PID=2943: ha llegado SIGALRM. Termino
PID=2952: ha llegado SIGALRM. Termino
PID=2961: ha llegado SIGALRM. Termino
PID=2940: mi hijo ha terminado. Yo termino
PID=2946: ha llegado SIGALRM. Termino
```

2. Justificación e Identificación de tareas

Dicho programa se ha diseñado a medida conforme al enunciado. Teniendo en cuenta de forma exhaustiva los requisitos de diseño e implementación de técnicas de programación en lenguaje C.

2.1. Recepción y validación de argumentos

Este programa comienza con la ejecución mediante el comando `./malla <X> <Y>` desde la terminal de Unix, tratándose de X de las

filas (Estructura horizontal) e Y de las columnas (Estructura vertical).

2.2. Generación de estructura malla

Al tratarse de un programa que crea una estructura variable, que depende de las variables X e Y introducidas como parámetro, la estructura debe de crearse mediante bucles for() y un sistema controlado de condicionales para evitar la aparición de “árboles inesperados”.

2.3. Estado pausado de la estructura final y ejecución de PSTREE

Al terminar la creación de todos y cada uno de sus hijos, los hijos se mantendrán durante un par de segundos en estado pausado. Estado que el padre aprovechará para ejecutar el programa “PSTREE” que muestra una estructura del árbol increíblemente precisa. Al terminar la ejecución de “PSTREE” el programa padre esperará a la muerte individual de todos y cada uno de sus hijos directos.

2.4. Llamada de alarma

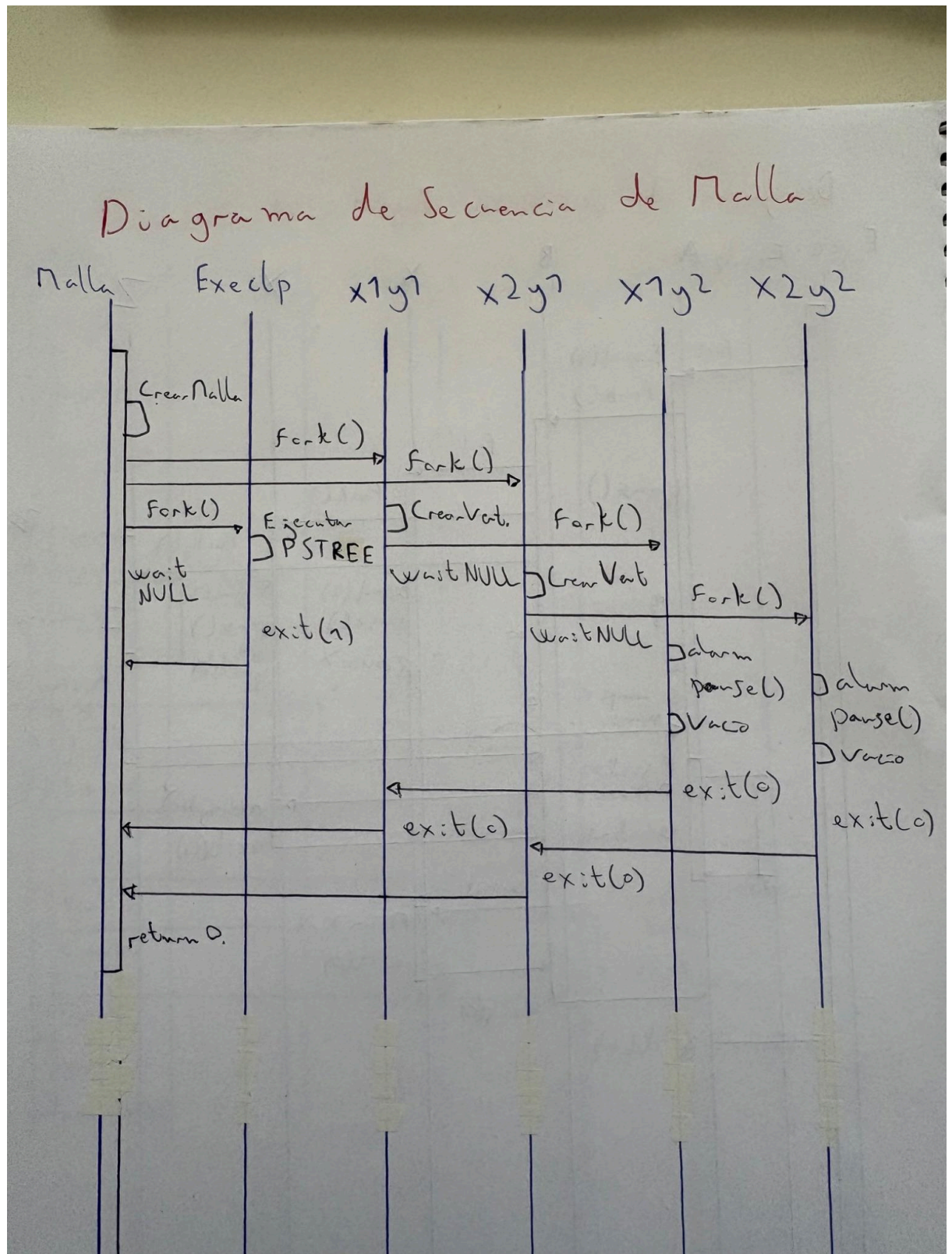
La llamada de alarma la ejecutará cada uno de los hijos finales de cada columna, de forma que debe de haberse creado dicha columna completa para la ejecución de la

alarma. Después de la ejecución de la alarma, esta finalizará la correcta ejecución de sus procesos hijos mediante "exit(0)".

2.5. Destrucción de la malla en cadena

Posteriormente todos y cada uno de los hijos y padres de cada columna se irán destruyendo en cadena mediante el uso de "wait(NULL)" seguido de "exit(0)", función que espera a que un hijo termine su ejecución para que el programa padre continúe y finalice.

3. Diagrama de Secuencia



4. Análisis detallado del código fuente

4.1. Funciones globales y de utilidad

4.1.1. Función Parse()

Esta función booleana es la que se encarga de la entrada y verificación de los datos introducidos por la terminal. Comprueba si se introduce la cantidad exacta de argumentos y que los valores son números enteros positivos. Finalmente devuelve true si se han pasado todos los filtros con éxito y false si no.

```
31 // Funcion que revisa que los argumentos sean correctos
32
33 bool parse(int argc, char *argv[])
34 {
35     if (argc != 3) {
36         printf("Uso: ./malla <x filas> <y columnas>\n");
37         return false;
38     }
39     else{
40         int x = atoi(argv[1]); /* numero de filas */
41         int y = atoi(argv[2]); /* numero de columnas */
42
43         if(x <= 0 || y <= 0){
44             printf("Error: los parametros deben de ser numeros enteros positivos")
45             return false;
46         }
47         else{
48             return true;
49         }
50     }
51 }
```

4.1.2. Función PSTREE()

Función de tipo void que ejecuta directamente el árbol de la malla "pstree()" mediante la llamada al sistema "execlp", que recibe los valores "-c" y el Pid del padre para que se limite a ejecutar el árbol

únicamente del proceso padre malla, y no muestre procesos externos ajenos al programa principal.

```
17 // Funcion PSTREE
18
19 void ejecutarPstree(){
20
21     printf("Arbol completo. Ejecuto pstree...\n");
22
23     char pidPadre[20];
24     sprintf(pidPadre, "%d", getppid());
25     execlp("pstree", "pstree", "-c", pidPadre, NULL);
26     perror("execlp");
27     exit(1);
28
29 }
```

4.2. Funciones de construcción y destrucción de la estructura

4.2.1. Función crearMalla()

Esta función recibe las medidas de las filas y las columnas (X e Y) junto con el PID del proceso actual. Comienza creando la estructura horizontal con las columnas que constituyen la primera fila.

Posteriormente comprueba si el pid que sigue por el programa es el hijo (`if (pid == 0)`) y crea la estructura vertical, si es el padre imprime por pantalla que ya se ha creado esa columna. Cuando se hayan creado todas las filas y columnas, el super padre intentará ejecutar el árbol PSTREE, y posteriormente hará un `wait(NULL)` por cada hijo (columna) que tenga.

Finalmente cuando haya pasado todos los `wait(NULL)` imprimirá por pantalla que todos sus hijos han terminado.

```

95
96 // ----- crearMalla: Crea la estructura Horizontal -----
97
98 void crearMalla(int x, int y, pid_t pid){
99
100     int col;
101     printf("P0: PID = %d, PPID = %d. Empiezo a crear columnas\n",
102           getpid(), getppid());
103
104     for (col = 1; col <= y; col++) {
105
106         pid = fork();
107
108         if (pid == 0) {
109
110             printf("PID = %d: soy la primera fila de la columna %d. Mi padre es %d\n",
111                   getpid(), col, getppid());
112
113             crearVertical(x, pid);
114         }
115         else{
116             printf("P0: he creado la columna %d, cuyo primer proceso es PID = %d\n",
117                   col, pid);
118         }
119     }
120
121     if(col == y + 1){
122
123         printf("P0: ya he creado todas las columnas. Espero a mis %d hijos \n",
124               y);
125
126         pid = fork();
127
128         if(pid == 0){
129
130             ejecutarPstree();
131         }
132
133         wait(NULL); // Espero al PSTREE
134
135         for(col = 1; col <= y; col++){
136
137             printf("P0: ejecutando wait numero %d\n",
138                   col);
139
140             wait(NULL);
141
142             printf("P0: uno de mis hijos directos ha terminado, %d \n",
143                   col);
144         }
145
146         printf("P0: todos mis hijos han terminado\n");
147     }
148
149 }
150

```

4.2.2. Función crearVertical()

Esta función recibe el número de filas y el PID del proceso actual. Nada más entrar comenzará un bucle for para crear todas las filas excepto la primera, que ya la creó la función anterior. Posteriormente hace que los PIDs padres hagan wait(NULL) esperando a sus hijos, después del wait hay un exit(0) para terminar el proceso nada más se despierten los padres. Finalmente cuando ya se han creado todas las filas, cada hijo ejecutará una alarma que apunta hacia la función de vacío, mientras los hijos se quedan en pause(), que únicamente sirve para despertar el proceso hijo de cada columna y que se muera con un exit(0).

```

53 // ---- CrearVertical: Crea la estructura vertical y ejecuta la alarma ----
54
55 void crearVertical(int x, pid_t pid){
56     int fila;
57
58     for(fila = 1; fila <= x - 1; fila++){
59
60         pid = fork();
61
62         if(pid != 0){
63
64             printf("PID = %d: he creado a mi hijo PID = %d. Espero con wait()\n",
65                 getpid(), pid);
66
67             wait(NULL);
68
69             printf("PID = %d: mi hijo ha terminado. Yo termino\n",
70                 getpid());
71
72             exit(0);
73         }
74
75         printf("PID = %d: continuo hacia la fila %d de mi columna\n",
76             getpid(), fila + 1);
77     }
78
79     if(fila == x){
80
81         printf("PID = %d: soy el ultimo de mi columna. Entro en pause()\n",
82             getpid());
83
84         signal(SIGALRM, vacio);
85         alarm(3);
86         pause();
87
88         printf("PID = %d: ha llegado SIGALRM. Termino\n",
89             getpid());
90
91         exit(0);
92     }
93 }
94

```

4.2.3. Función de vacio()

Como ya hemos dicho antes, esta función únicamente sirve para despertar a los últimos hijos del `pause()`, no recibe ni devuelve ningún dato.

4.3. Función principal de malla

4.3.1. Función main()

Esta es la función principal del programa y en este caso es bastante simple y sencilla. Recibe los argumentos introducidos en la terminal que posteriormente se los pasará a la función Parse() que comprueba los argumentos. Si los argumentos no son correctos, mostrará un mensaje por pantalla. En cambio si son correctos creará las variables X e Y en las que guardará los valores introducidos en la terminal y ejecutará la función crearMalla, que se encargará de la creación y destrucción de la malla. Finalmente se mostrará el mensaje "He terminado el programa con éxito". Esta función únicamente la ejecutará el super padre ya que los hijos morirán antes de volver a ella.

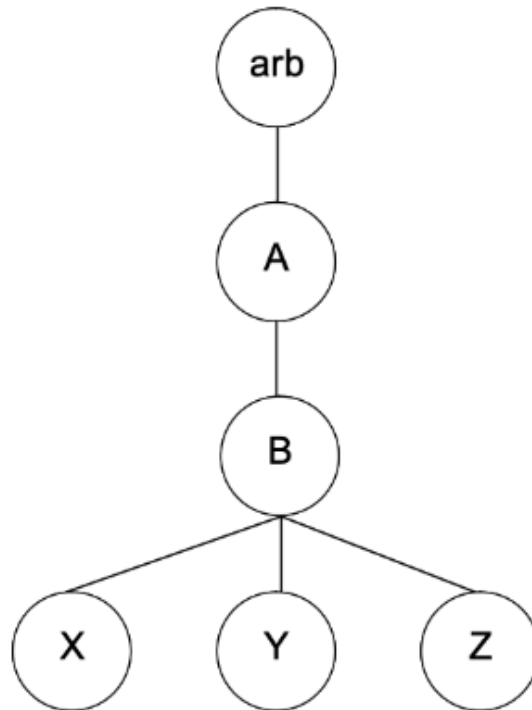
```
151 // ----- Main: Malla principal -----
152
153 int main(int argc, char *argv[])
154 {
155     if (!parse(argc, argv)) {
156         printf("Error: los argumentos deben ser 3 y solo admiten enteros positivos.\n");
157         exit(1);
158     }
159     else{
160         int x, y;
161         pid_t pid;
162
163         x = atoi(argv[1]);
164         y = atoi(argv[2]);
165
166         crearMalla(x, y, pid);
167     }
168
169     printf("Soy el programa %d y he terminado con exito", getpid());
170
171     return 0;
172 }
```

2. Ejercicio 1 B

1. Enunciado del ejercicio

Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando “pstree”. El proceso Z no puede utilizar el comando “sleep”, por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.1. Estructura del Árbol de Procesos



1.2. Resultado Esperado

1.2.1. Ejemplo 1: 2 segundos

```
Soy ejec (3042) y muero
ubuntu@ubuntu:~/Documents/SO/P1/Ejercicio1/Solucion$ ./ejec 2
Soy el proceso ejec: mi pid es 3055
Soy el proceso A: mi pid es 3056. Mi padre es 3055
Soy el proceso B: mi pid es 3057. Mi padre es 3056. Mi abuelo es 3055
Soy el proceso X: mi pid es 3058. Mi padre es 3057. Mi abuelo es 3056. Mi bisabuelo es 3055
Soy el proceso Y: mi pid es 3059. Mi padre es 3057. Mi abuelo es 3056. Mi bisabuelo es 3055
Soy el proceso Z: mi pid es 3060. Mi padre es 3057. Mi abuelo es 3056. Mi bisabuelo es 3055
Arbol completo. Ejecuto pstree...
ejec--ejec--ejec
      |   |--ejec
      |   |--ejec
      |   --pstree
Soy Z (3060) y muero
Soy Y (3059) y muero
Soy X (3058) y muero
Soy B (3057) y muero
Soy A (3056) y muero
Soy ejec (3055) y muero
```

1.2.2. Ejemplo 2: 4 segundos

```
ubuntu@ubuntu:~/Documents/SO/P1/Ejercicio1/Solucion$ ./ejec 4
Soy el proceso ejec: mi pid es 3062
Soy el proceso A: mi pid es 3063. Mi padre es 3062
Soy el proceso B: mi pid es 3064. Mi padre es 3063. Mi abuelo es 3062
Soy el proceso X: mi pid es 3065. Mi padre es 3064. Mi abuelo es 3063. Mi bisabuelo es 3062
Soy el proceso Y: mi pid es 3066. Mi padre es 3064. Mi abuelo es 3063. Mi bisabuelo es 3062
Soy el proceso Z: mi pid es 3067. Mi padre es 3064. Mi abuelo es 3063. Mi bisabuelo es 3062
Arbol completo. Ejecuto pstree...
ejec--ejec--ejec
      |   |   |
      |   |   +--ejec
      |   |   +--ejec
      |   |
      |   +---pstree
      |
      +-----
```

```
Soy Z (3067) y muero
Soy Y (3066) y muero
Soy X (3065) y muero
Soy B (3064) y muero
Soy A (3063) y muero
Soy ejec (3062) y muero
```

2. Justificación e Identificación de Tareas

Este programa se ha diseñado conforme a las especificaciones y necesidades dadas en el enunciado.

2.1. Recepción y validación de argumentos

El programa comienza por la ejecución en terminal mediante la llamada `./ejec <segundos>`, siendo `<segundos>` la cantidad de tiempo que tardará el hijo Z en enviar la señal al hijo A.

2.2. Creación del árbol de procesos

Tras introducir el comando por terminal, se irán creando los procesos Árbol, A, B, X, Y y Z. Cada proceso contiene su propia función.

2.3. Temporización de la alarma

Tras la creación del proceso Z, este iniciará la alarma con los segundos que se introdujeron por pantalla. Pasado el tiempo el proceso Z enviará una llamada kill al proceso A mientras Z espera a la alarma.

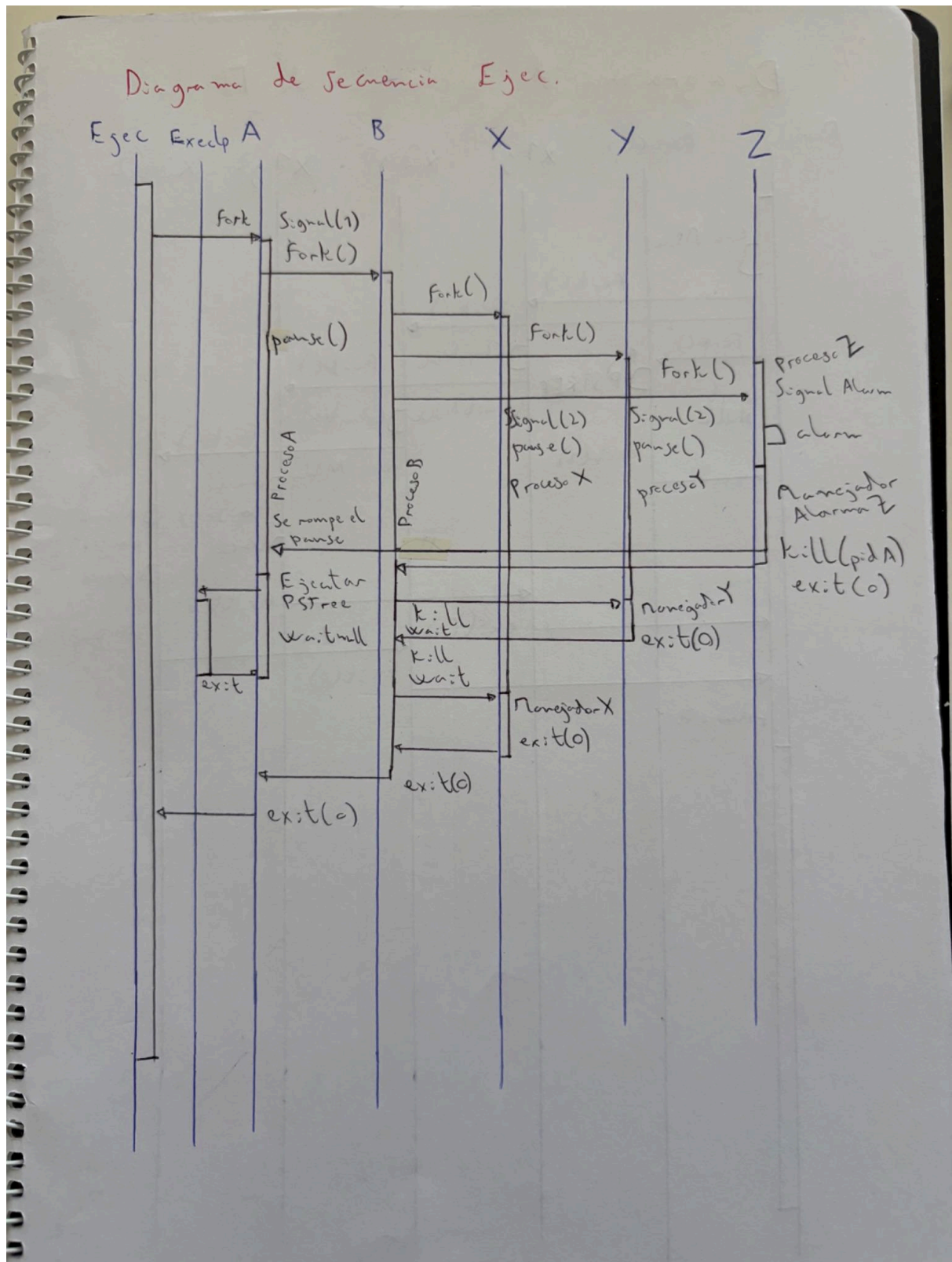
2.4. Recepción y ejecución de manejadores de señales

Cada proceso tiene manejadores de señales, excepto B que no lo necesita, estos manejadores gestionan las señales alarm y sigusr2.

2.5. Destrucción progresiva y controlada de procesos

Esto provoca que X, Y y Z se mueran los primeros, mientras que B espera la muerte de ellos y A, a su vez, espera la muerte de B.

3. Diagrama de Secuencia



4. Análisis detallado del Código fuente

4.1. Funciones globales y de utilidad

4.1.1. Variables Globales

Este programa usa dos variables globales, una para almacenar el PID de A, y otra para hacer despertar al proceso A cuando el hijo Z le envía la señal de muerte.

```
/* Variables globales: los manejadores de señal solo reciben el
   numero de señal como parametro, asi que para que puedan usar
   otros datos (como el PID al que hay que avisar) necesitamos
   variables globales. */
pid_t pidA;           // PID de A, lo usa el manejador de Z
int senalRecibida = 0; // la usa A para saber si ya le llego la señal
```

4.1.2. ejecutarPstree

Función que ejecuta la llamada al sistema que muestra el árbol de procesos por pantalla.

```

18 // Funcion de PSTREE
19
20 void ejecutarPstree(pid_t pid){
21
22     printf("Arbol completo. Ejecuto pstree...\n");
23
24     char pidt[20];
25     sprintf(pidt, "%d", pid);
26     execlp("pstree", "pstree", "-c", pidt, NULL);
27     perror("execlp");
28     exit(1);
29 }
30

```

4.2. Funciones de construcción del árbol

4.2.1. ProcesoA

Esta función comienza recogiendo su PID para guardarlo en la variable global "pidA", que posteriormente usará el proceso Z para enviar la señal de muerte. Posteriormente configuramos mediante `signal()` la señal `SIGUSR1` para que nos envíe a la función `manejadorUSR1A`. Después crea a su hijo B y llama a la función B para que solamente la ejecute dicho hijo, mientras el proceso A se queda pausado en el `While()` que ejecuta un `pause()` mientras la variable global `senalRecibida` sea 0. Cuando el hijo Z muera, este llamará al proceso A para que ejecute el `manejadorUSR1A`, cambiando el valor de `senalRecibida` y continuando la ejecución de la función. Esto hará que el proceso A pueda ejecutar el árbol `pstree` mientras espera a la muerte del árbol y del proceso B. Finalmente el proceso A muere con un `exit(0)`.

```

121 // ----- Proceso A -----
122 void procesoA(pid_t pidEjec, int segundos){
123     int miPid = getpid();
124     printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", miPid, pidEjec);
125
126     signal(SIGUSR1, manejadorUsr1A);
127
128     int pidB = fork();
129     if (pidB == 0) {
130         procesoB(pidEjec, miPid, segundos);
131         exit(0);
132     }
133
134     while (senalRecibida == 0) {
135         pause(); // espero la señal de Z
136     }
137
138     /* Creo un hijo para que ejecute pstree (asi yo puedo seguir
139     vivo despues y esperar correctamente a B) */
140
141     int pidPstree = fork();
142
143     if (pidPstree == 0){
144         ejecutarPstree(miPid);
145     }
146
147     wait(NULL); // espero a que termine de mostrarse el pstree
148
149     wait(NULL); // espero a que B (y todo su subarbol) haya muerto
150
151     printf("Soy A (%d) y muero\n", miPid);
152     exit(0);
153 }

```

4.2.2. ProcesoB

La función del proceso B comienza recogiendo y guardando su PID. Posteriormente crea y ejecuta los procesos hijos X, Y y Z, guardando sus PIDs en variables independientes. Cuando ya están todos sus hijos creados, el proceso B espera a la muerte de Z, posteriormente envía una señal de muerte al proceso Y y finalmente a X. Cuando todos sus hijos han muerto, el proceso B muere mostrando un mensaje por pantalla.

```

83 // ----- Proceso B -----
84 void procesoB(pid_t pidEjec, pid_t pidPadre, int segundos){
85
86     pid_t miPid = getpid();
87     printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n",
88         miPid, pidPadre, pidEjec);
89
90     pid_t pidX = fork();
91     if (pidX == 0) {
92         procesoX(pidEjec, pidPadre, miPid);
93     }
94
95     pid_t pidY = fork();
96     if (pidY == 0) {
97         procesoY(pidEjec, pidPadre, miPid);
98     }
99
100    pid_t pidZ = fork();
101    if (pidZ == 0) {
102        procesoZ(pidEjec, pidPadre, miPid, segundos);
103    }
104
105    /* En este momento X e Y solo estan en pause(), nada los puede
106       despertar todavia, asi que el UNICO hijo que puede morir es
107       Z (cuando salte su alarma). Por eso un wait() normal ya
108       reúne el orden correcto sin necesidad de nada mas avanzado. */
109    wait(NULL);          /* espero a que muera Z (Hijo) */
110
111    kill(pidY, SIGUSR2); /* ahora aviso a Y */
112    wait(NULL);          /* y espero a que muera */
113
114    kill(pidX, SIGUSR2); /* despues aviso a X */
115    wait(NULL);          /* y espero a que muera */
116
117    printf("Soy B (%d) y muero\n", miPid);
118    exit(0);
119 }

```

4.2.3. ProcesoX e Y

Estos procesos son idénticos. Imprimen un mensaje por pantalla informando su creación, configuran la señal de muerte SIGUSR2, y se quedan en `pause()` esperando la llamada `kill` del proceso B.

```

53 // Proceso X
54 void procesoX(pid_t pidEjec, pid_t pidAbuelo, pid_t pidPadre){
55     printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n",
56           getpid(), pidPadre, pidAbuelo, pidEjec);
57
58     signal(SIGUSR2, manejadorUsr2X);
59     pause(); /* espero a que B me avise */
60 }

```

```

62 // Proceso Y
63 void procesoY(pid_t pidEjec, pid_t pidAbuelo, pid_t pidPadre){
64     printf("Soy el proceso Y: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n",
65           getpid(), pidPadre, pidAbuelo, pidEjec);
66
67     signal(SIGUSR2, manejadorUsr2Y);
68     pause();
69 }
70

```

4.2.4. ProcesoZ

Este varía un poco con respecto a sus hermanos. Imprime un mensaje informando de su creación por pantalla, recoge el PID de su abuelo A para que lo pueda usar el manejador para enviar la señal de muerte. Después configura la señal de muerte de sí mismo con SIGALRM y ejecuta la alarma mientras se queda en `pause()`, esta alarma hace que el proceso Z ejecute la función de su manejador, provocando su muerte.

```

71 // Proceso Z
72 void procesoZ(pid_t pidEjec, pid_t pidAbuelo, pid_t pidPadre, int segundos){
73     printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n",
74           getpid(), pidPadre, pidAbuelo, pidEjec);
75
76     pidA = pidAbuelo; /* guardo el PID de A para que lo use el manejador */
77     kill(pidA, SIGUSR1);
78     signal(SIGALRM, manejadorAlarmaZ);
79     alarm(segundos);
80     pause();
81 }
82

```

4.3. Manejadores de señal

4.3.1. ManejadorUsr1A

Únicamente cambia el valor de la variable global `senalRecibida` de 0 a 1 para que el proceso A salga de la espera y ejecute el `PSTREE`.

```
33 void manejadorUsr1A(){  
34     senalRecibida = 1;  
35 }
```

4.3.2. ManejadorAlarmaZ, ManejadorUsr2Y y ManejadorUsr2X

Únicamente hacen que terminen los procesos hijos X e Y mediante `exit(0)` y que muestren un mensaje de muerte por pantalla.

```
37 void manejadorAlarmaZ(){  
38  
39     printf("Soy Z (%d) y muero\n", getpid());  
40     exit(0);  
41 }  
42  
43 void manejadorUsr2X(){  
44     printf("Soy X (%d) y muero\n", getpid());  
45     exit(0);  
46 }  
47  
48 void manejadorUsr2Y(){  
49     printf("Soy Y (%d) y muero\n", getpid());  
50     exit(0);  
51 }  
52
```

4.4. Función principal

4.4.1. Función Main()

La función principal se encarga de procesar los argumentos, posteriormente crea a su hijo A mediante un `fork()` y espera la muerte de este mediante un `wait(NULL)`, cuando muere A, el proceso principal vuelve al funcionamiento que mostrará un mensaje por pantalla y terminará el programa.

```
155 // ----- main = proceso ejec (arb) -----
156 int main(int argc, char *argv[]){
157
158     if (argc != 2 || argv[1] < 0) {
159         printf("Uso: ./ejec <segundos>\n");
160         exit(1);
161     }
162
163     int segundos = atoi(argv[1]);
164     int pidEjec = getpid();
165
166     printf("Soy el proceso ejec: mi pid es %d\n", pidEjec);
167
168     pid_t pidA_hijo = fork();
169     if (pidA_hijo == 0) {
170         procesoA(pidEjec, segundos);
171         exit(0);
172     }
173
174     wait(NULL); // espero a que A (y todo el arbol) haya muerto
175     printf("Soy ejec (%d) y muero\n", pidEjec);
176     return 0;
177 }
```

3. Ejercicio 2

1. Enunciado del ejercicio

Realizar un programa llamado **hacha.c** que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, Se debe tener en cuenta el formato y consideraciones siguientes:

\$ hacha <archivo> <tamaño>

<archivo> nombre del archivo a dividir

<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

1.1. Objetivo del programa

Este programa se ha hecho con el fin de coger un archivo, da igual el tipo que sea, y lo dividimos en pequeños trozos del tamaño indicado por terminal utilizando tuberías.

1.2. Resultado esperado

1.2.1. Ejemplo 1: capturaOriginal.png en pedazos de 50000 Bytes

```
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejec$ ls
capturaOriginal.png hacha
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejec$ ./hacha capturaOriginal.png 100000
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejec$ ls
capturaOriginal.png  capturaOriginal.png.h10  capturaOriginal.png.h13  capturaOriginal.png.h3  capturaOriginal.png.h6  capturaOriginal.png.h9
capturaOriginal.png.h0  capturaOriginal.png.h11  capturaOriginal.png.h14  capturaOriginal.png.h4  capturaOriginal.png.h7  hacha
capturaOriginal.png.h1  capturaOriginal.png.h12  capturaOriginal.png.h2   capturaOriginal.png.h5  capturaOriginal.png.h8
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejec$
```

1.2.2. Ejemplo 2: VSCode.deb en pedazos de 500000 Bytes

```
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejet$ ls
hacha  vscode.deb
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejet$ ./hacha vscode.deb 500000
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejet$ ls
hacha  vscode.deb.h140  vscode.deb.h185  vscode.deb.h229  vscode.deb.h273  vscode.deb.h317  vscode.deb.h361  vscode.deb.h405  vscode.deb.h55
vscode.deb  vscode.deb.h141  vscode.deb.h186  vscode.deb.h23  vscode.deb.h274  vscode.deb.h318  vscode.deb.h362  vscode.deb.h406  vscode.deb.h56
vscode.deb.h0  vscode.deb.h142  vscode.deb.h187  vscode.deb.h230  vscode.deb.h275  vscode.deb.h319  vscode.deb.h363  vscode.deb.h407  vscode.deb.h57
vscode.deb.h1  vscode.deb.h143  vscode.deb.h188  vscode.deb.h231  vscode.deb.h276  vscode.deb.h32  vscode.deb.h364  vscode.deb.h408  vscode.deb.h58
vscode.deb.h10  vscode.deb.h144  vscode.deb.h189  vscode.deb.h232  vscode.deb.h277  vscode.deb.h320  vscode.deb.h365  vscode.deb.h409  vscode.deb.h59
vscode.deb.h100  vscode.deb.h145  vscode.deb.h19  vscode.deb.h233  vscode.deb.h278  vscode.deb.h321  vscode.deb.h366  vscode.deb.h41  vscode.deb.h6
vscode.deb.h101  vscode.deb.h146  vscode.deb.h190  vscode.deb.h234  vscode.deb.h279  vscode.deb.h322  vscode.deb.h367  vscode.deb.h410  vscode.deb.h60
vscode.deb.h102  vscode.deb.h147  vscode.deb.h191  vscode.deb.h235  vscode.deb.h28  vscode.deb.h323  vscode.deb.h368  vscode.deb.h411  vscode.deb.h61
vscode.deb.h103  vscode.deb.h148  vscode.deb.h192  vscode.deb.h236  vscode.deb.h280  vscode.deb.h324  vscode.deb.h369  vscode.deb.h412  vscode.deb.h62
vscode.deb.h104  vscode.deb.h149  vscode.deb.h193  vscode.deb.h237  vscode.deb.h281  vscode.deb.h325  vscode.deb.h37  vscode.deb.h413  vscode.deb.h63
vscode.deb.h105  vscode.deb.h15  vscode.deb.h194  vscode.deb.h238  vscode.deb.h282  vscode.deb.h326  vscode.deb.h370  vscode.deb.h414  vscode.deb.h64
vscode.deb.h106  vscode.deb.h150  vscode.deb.h195  vscode.deb.h239  vscode.deb.h283  vscode.deb.h327  vscode.deb.h371  vscode.deb.h415  vscode.deb.h65
vscode.deb.h107  vscode.deb.h151  vscode.deb.h196  vscode.deb.h24  vscode.deb.h284  vscode.deb.h328  vscode.deb.h372  vscode.deb.h416  vscode.deb.h66
vscode.deb.h108  vscode.deb.h152  vscode.deb.h197  vscode.deb.h240  vscode.deb.h285  vscode.deb.h329  vscode.deb.h373  vscode.deb.h417  vscode.deb.h67
vscode.deb.h109  vscode.deb.h153  vscode.deb.h198  vscode.deb.h241  vscode.deb.h286  vscode.deb.h33  vscode.deb.h374  vscode.deb.h418  vscode.deb.h68
vscode.deb.h11  vscode.deb.h154  vscode.deb.h199  vscode.deb.h242  vscode.deb.h287  vscode.deb.h330  vscode.deb.h375  vscode.deb.h419  vscode.deb.h69
vscode.deb.h110  vscode.deb.h155  vscode.deb.h2  vscode.deb.h243  vscode.deb.h288  vscode.deb.h331  vscode.deb.h376  vscode.deb.h42  vscode.deb.h7
vscode.deb.h111  vscode.deb.h156  vscode.deb.h20  vscode.deb.h244  vscode.deb.h289  vscode.deb.h332  vscode.deb.h377  vscode.deb.h420  vscode.deb.h70
vscode.deb.h112  vscode.deb.h157  vscode.deb.h200  vscode.deb.h245  vscode.deb.h29  vscode.deb.h333  vscode.deb.h378  vscode.deb.h421  vscode.deb.h71
vscode.deb.h113  vscode.deb.h158  vscode.deb.h201  vscode.deb.h246  vscode.deb.h290  vscode.deb.h334  vscode.deb.h379  vscode.deb.h422  vscode.deb.h72
vscode.deb.h114  vscode.deb.h159  vscode.deb.h202  vscode.deb.h247  vscode.deb.h291  vscode.deb.h335  vscode.deb.h38  vscode.deb.h423  vscode.deb.h73
vscode.deb.h115  vscode.deb.h16  vscode.deb.h203  vscode.deb.h248  vscode.deb.h292  vscode.deb.h336  vscode.deb.h380  vscode.deb.h424  vscode.deb.h74
vscode.deb.h116  vscode.deb.h160  vscode.deb.h204  vscode.deb.h249  vscode.deb.h293  vscode.deb.h337  vscode.deb.h381  vscode.deb.h425  vscode.deb.h75
vscode.deb.h117  vscode.deb.h161  vscode.deb.h205  vscode.deb.h25  vscode.deb.h294  vscode.deb.h338  vscode.deb.h382  vscode.deb.h426  vscode.deb.h76
vscode.deb.h118  vscode.deb.h162  vscode.deb.h206  vscode.deb.h250  vscode.deb.h295  vscode.deb.h339  vscode.deb.h383  vscode.deb.h427  vscode.deb.h77
vscode.deb.h119  vscode.deb.h163  vscode.deb.h207  vscode.deb.h251  vscode.deb.h296  vscode.deb.h34  vscode.deb.h384  vscode.deb.h428  vscode.deb.h78
vscode.deb.h12  vscode.deb.h164  vscode.deb.h208  vscode.deb.h252  vscode.deb.h297  vscode.deb.h340  vscode.deb.h385  vscode.deb.h429  vscode.deb.h79
vscode.deb.h120  vscode.deb.h165  vscode.deb.h209  vscode.deb.h253  vscode.deb.h298  vscode.deb.h341  vscode.deb.h386  vscode.deb.h43  vscode.deb.h8
vscode.deb.h121  vscode.deb.h166  vscode.deb.h21  vscode.deb.h254  vscode.deb.h299  vscode.deb.h342  vscode.deb.h387  vscode.deb.h430  vscode.deb.h80
vscode.deb.h122  vscode.deb.h167  vscode.deb.h210  vscode.deb.h255  vscode.deb.h3  vscode.deb.h343  vscode.deb.h388  vscode.deb.h431  vscode.deb.h81
vscode.deb.h123  vscode.deb.h168  vscode.deb.h211  vscode.deb.h256  vscode.deb.h30  vscode.deb.h344  vscode.deb.h389  vscode.deb.h432  vscode.deb.h82
vscode.deb.h124  vscode.deb.h169  vscode.deb.h212  vscode.deb.h257  vscode.deb.h300  vscode.deb.h345  vscode.deb.h39  vscode.deb.h433  vscode.deb.h83
vscode.deb.h125  vscode.deb.h17  vscode.deb.h213  vscode.deb.h258  vscode.deb.h301  vscode.deb.h346  vscode.deb.h390  vscode.deb.h434  vscode.deb.h84
vscode.deb.h126  vscode.deb.h170  vscode.deb.h214  vscode.deb.h259  vscode.deb.h302  vscode.deb.h347  vscode.deb.h391  vscode.deb.h435  vscode.deb.h85
vscode.deb.h127  vscode.deb.h171  vscode.deb.h215  vscode.deb.h26  vscode.deb.h303  vscode.deb.h348  vscode.deb.h392  vscode.deb.h436  vscode.deb.h86
vscode.deb.h128  vscode.deb.h172  vscode.deb.h216  vscode.deb.h260  vscode.deb.h304  vscode.deb.h349  vscode.deb.h393  vscode.deb.h437  vscode.deb.h87
vscode.deb.h129  vscode.deb.h173  vscode.deb.h217  vscode.deb.h261  vscode.deb.h305  vscode.deb.h35  vscode.deb.h394  vscode.deb.h438  vscode.deb.h88
vscode.deb.h13  vscode.deb.h174  vscode.deb.h218  vscode.deb.h262  vscode.deb.h306  vscode.deb.h350  vscode.deb.h395  vscode.deb.h44  vscode.deb.h89
vscode.deb.h130  vscode.deb.h175  vscode.deb.h219  vscode.deb.h263  vscode.deb.h307  vscode.deb.h351  vscode.deb.h396  vscode.deb.h45  vscode.deb.h9
vscode.deb.h131  vscode.deb.h176  vscode.deb.h22  vscode.deb.h264  vscode.deb.h308  vscode.deb.h352  vscode.deb.h397  vscode.deb.h46  vscode.deb.h90
vscode.deb.h132  vscode.deb.h177  vscode.deb.h220  vscode.deb.h265  vscode.deb.h309  vscode.deb.h353  vscode.deb.h398  vscode.deb.h47  vscode.deb.h91
vscode.deb.h133  vscode.deb.h178  vscode.deb.h221  vscode.deb.h266  vscode.deb.h31  vscode.deb.h354  vscode.deb.h399  vscode.deb.h48  vscode.deb.h92
vscode.deb.h134  vscode.deb.h179  vscode.deb.h222  vscode.deb.h267  vscode.deb.h310  vscode.deb.h355  vscode.deb.h4  vscode.deb.h49  vscode.deb.h93
vscode.deb.h135  vscode.deb.h18  vscode.deb.h223  vscode.deb.h268  vscode.deb.h311  vscode.deb.h356  vscode.deb.h40  vscode.deb.h5  vscode.deb.h94
vscode.deb.h136  vscode.deb.h180  vscode.deb.h224  vscode.deb.h269  vscode.deb.h312  vscode.deb.h357  vscode.deb.h400  vscode.deb.h50  vscode.deb.h95
vscode.deb.h137  vscode.deb.h181  vscode.deb.h225  vscode.deb.h27  vscode.deb.h313  vscode.deb.h358  vscode.deb.h401  vscode.deb.h51  vscode.deb.h96
vscode.deb.h138  vscode.deb.h182  vscode.deb.h226  vscode.deb.h270  vscode.deb.h314  vscode.deb.h359  vscode.deb.h402  vscode.deb.h52  vscode.deb.h97
vscode.deb.h139  vscode.deb.h183  vscode.deb.h227  vscode.deb.h271  vscode.deb.h315  vscode.deb.h36  vscode.deb.h403  vscode.deb.h53  vscode.deb.h98
vscode.deb.h14  vscode.deb.h184  vscode.deb.h228  vscode.deb.h272  vscode.deb.h316  vscode.deb.h360  vscode.deb.h404  vscode.deb.h54  vscode.deb.h99
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicios/Solejet$
```

2. Justificación e Identificación de Tareas

2.1. Recepción y procesamiento de argumentos

Escribimos por terminal el comando `./hacha <nombre del archivo> <tamaño de cada trozo>`, si no es correcto se mostrará un mensaje de error y se terminará la ejecución del programa. En cambio, si se han introducido correctamente los argumentos, se procederá a la correcta ejecución del programa.

2.2. Planteamiento de la partición y del número de hijos

Primero calculamos el número de hijos necesarios en función del tamaño del archivo y de los trozos a realizar. Después se irá creando cada uno de los hijos para que cree cada uno de los trozos.

2.3. Tareas del Padre

2.3.1. Lectura del archivo

El padre procede a leer los bytes del archivo que escribirá cada uno de sus hijos, en función del tamaño del trozo.

2.3.2. Escritura en tubería

Después de leer cada trozo del archivo principal, procede a escribirlo en la tubería en la segunda sección, la dedicada a la escritura. Cuando termina la escritura de cada trozo, cierra la tubería para que no se generen problemas con futuros hijos.

2.3.3. Espera a la muerte de sus hijos

Al terminar con la escritura completa de todo el archivo en la tubería, el padre deberá esperar a la muerte de todos y cada uno de sus hijos para continuar con la ejecución del programa. Hemos de controlar muy bien la supervivencia del padre para que no se nos queden niños huérfanos.

2.4. Tareas de los Hijos

2.4.1. Lectura del segmento de tubería

Cada hijo lee el segmento de la primera sección de la tubería que le corresponde. Segmento que acaba de escribir el padre.

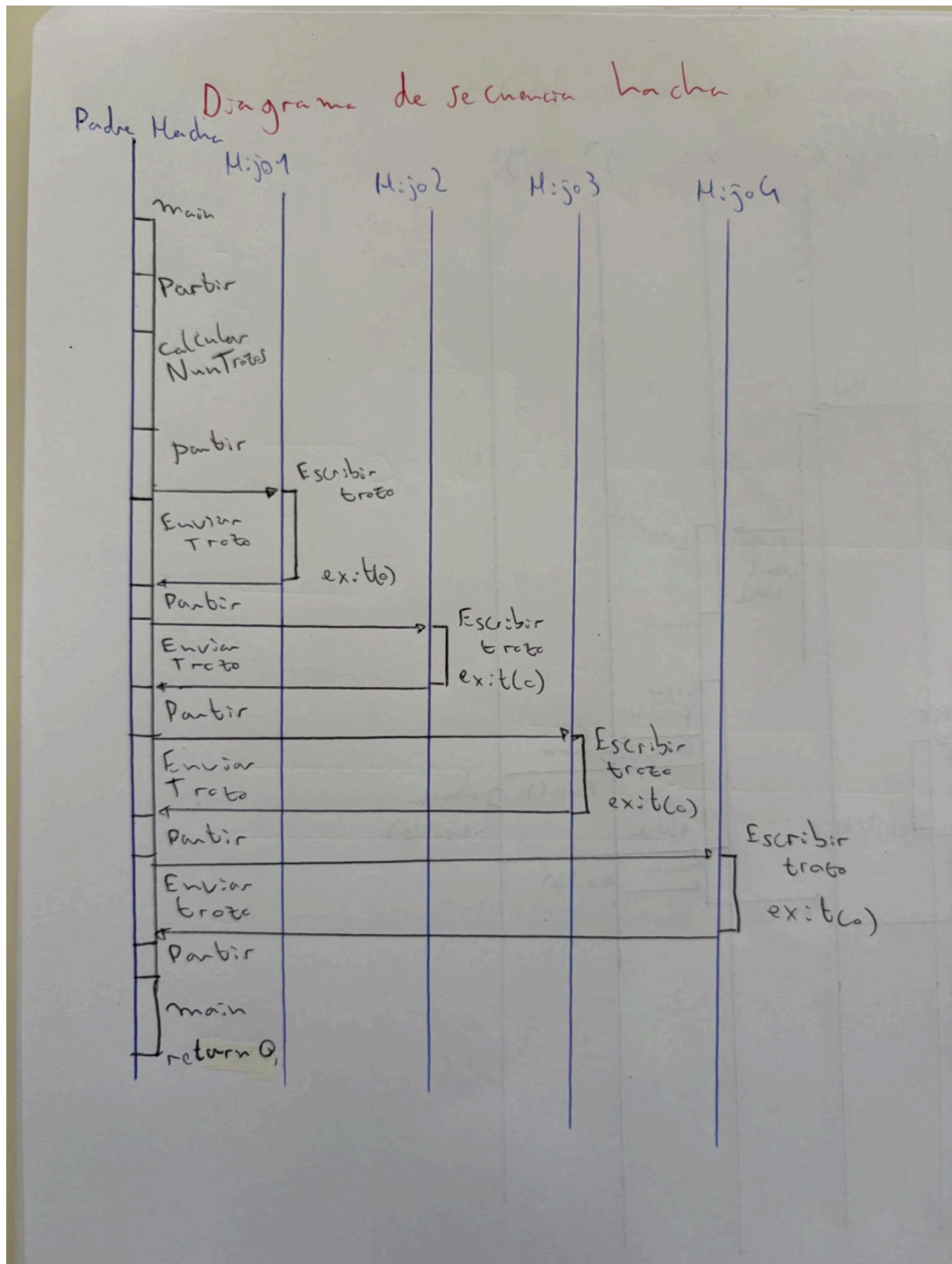
2.4.2. Creación y escritura de los trozos

Posteriormente comienza a realizar cada uno de los trozos en los que se divide el archivo principal, y a escribir la parte de memoria que le corresponde.

2.4.3. Plácida muerte

Tras la correcta creación del trozo correspondiente, el hijo cierra el archivo de salida del trozo, y seguidamente el primer segmento de la tubería dedicado a la lectura. Hecha su función el hijo procederá a morir plácidamente.

3. Diagrama de Secuencia



4. Análisis detallado del Código fuente

4.1. Función partir()

Esta función recibe como parámetros de entrada el nombre del archivo con un puntero de char, y el tamaño de cada trozo del archivo principal.

Comienza con la apertura del archivo introducido como parámetro en modo Solo-Lectura mediante la llamada `open(archivo, O_RDONLY)`. Posteriormente se creará una variable struct stat que se utilizará para extraer el tamaño total del archivo. Después calculamos el número de trozos mediante la llamada a la función `calcularNumTrozos()`, y continuamos con la reserva de memoria dedicada a los trozos que se almacenarán en la tubería. Creamos la variable 'tubería' que se destinará a cada tubería.

Posteriormente entramos en el bucle for que crea la la tubería y el hijo que la leerá. En este punto el padre llamará a la función `enviarTrozo()` mientras el hijo llama a la función `escribirTrozo()`. Hecho esto se vaciará el segmento de memoria que usa el hijo para que lo use el siguiente hijo y este morirá. Terminado este primer for, el proceso padre entrará en un segundo for que hará tantos `wait(NULL)` como hijos haya generado, para controlar la supervivencia del padre frente a la muerte de sus hijos. Finalmente se cierra el archivo leído con `close(fEntrada)` y se libera el segmento de memoria que usó el padre.

```

65 // Parte el archivo "nombre" en trozos de "tamanyo" bytes, creando un hijo por
66 // cada trozo. Cada trozo usa un tubo distinto.
67 void partir(char *nombre, int tamanyo){
68
69     int fEntrada = open(nombre, O_RDONLY);
70
71     if(fEntrada < 0){
72
73         printf("Error. No existe el fichero\n");
74     }
75     else{
76
77         // stat rellena propFichero con la informacion del archivo; aqui solo se
78         // usa su tamaño total en bytes (st_size).
79         struct stat propFichero;
80         stat(nombre, &propFichero);
81
82         int numTrozos = calcularNumTrozos(propFichero.st_size, tamanyo);
83
84         // Memoria donde se guarda cada trozo mientras pasa por el tubo.
85         // Tras el fork, padre e hijo tienen cada uno su propia copia.
86         char *buffer = (char *) malloc(sizeof(char) * tamanyo);
87
88         int tuberia[2]; // tuberia[0] = lectura, tuberia[1] = escritura
89
90         for(int i = 0; i < numTrozos; i++){
91
92             pipe(tuberia);
93
94             if(fork() == 0){
95                 // Proceso hijo: escribe su trozo y termina aqui, para que no
96                 // siga ejecutando el bucle y creando mas hijos.
97                 escribirTrozo(tuberia[0], tuberia[1], nombre, i, tamanyo, buffer);
98                 free(buffer);
99                 exit(0);
100             }
101
102             // Proceso padre: manda el trozo y pasa a crear el siguiente hijo.
103             enviarTrozo(fEntrada, tuberia[0], tuberia[1], buffer, tamanyo);
104         }
105
106         // El padre espera a que terminen todos los hijos antes de acabar.
107         for(int i = 0; i < numTrozos; i++){
108             wait(NULL);
109         }
110
111         close(fEntrada);
112         free(buffer);
113     }
114 }
115

```

4.2. Función calcularNumTrozos()

Recibe por parámetros el tamaño total del archivo y el tamaño de cada trozo, así que realiza la división para calcular el número de trozos, y en caso de que la división no sea exacta, se añadirá un trozo adicional.

```

15  int calcularNumTrozos(long tamTotal, int tamanyo){
16
17      int numTrozos = tamTotal / tamanyo;
18
19      if(tamTotal % tamanyo != 0){
20          numTrozos++;
21      }
22      return numTrozos;
23  }

```

4.3. Función escribirTrozo()

Esta función recibe por parámetros el pipe de la lectura y escritura, el nombre del archivo, el número de trozo correspondiente, el tamaño de cada trozo y el segmento de memoria de donde extraerá el contenido. Comienza creando el fichero de salida con `creat()`. Ahora usamos un bucle `while` para recorrer el contenido de la tubería hasta que devuelva cero, cuando el padre haya terminado la lectura de este trozo, mientras escribimos en el archivo de entrada del trozo. Finalmente cerramos el trozo y el segmento de tubería utilizado para la lectura.

```

27 void escribirTrozo(int lectura, int escritura, char *nombre, int i, int tamanyo, char *buffer){
28
29     // El hijo solo lee del tubo, asi que cierra su copia del extremo de
30     // escritura. Si no lo cerrara, read() nunca devolveria 0 (fin de datos),
31     // porque el propio hijo seguiria contando como "escritor" del tubo.
32     close(escritura);
33
34     char nombreFichero[256];
35     sprintf(nombreFichero, "%s.h%d", nombre, i + 1);
36     int fSalida = creat(nombreFichero, 0666);
37
38     // Un tubo puede entregar los datos en varias partes, asi que se repite
39     // read() hasta que devuelva 0: eso ocurre cuando el tubo esta vacio y el
40     // padre ya ha cerrado su extremo de escritura.
41     int numLeidos;
42     while((numLeidos = read(lectura, buffer, tamanyo)) > 0){
43         write(fSalida, buffer, numLeidos);
44     }
45
46     close(fSalida);
47     close(lectura);
48 }

```

4.4. Función enviarTrozo()

Recibe como parámetro el archivo de entrada, el pipe de lectura y escritura, el segmento de memoria para transferir los trozos del archivo al pipe, y el tamaño de cada trozo. Comenzamos cerrando el pipe de lectura porque no lo vamos a utilizar, después guardamos en el buffer el contenido leído y en numLeidos el número de bytes leídos. Después escribimos en el segmento de escritura del pipe el contenido de la variable buffer teniendo como límite el número de bytes leídos y guardados en numLeidos. Finalmente cerramos el pipe de escritura.

```

52 void enviarTrozo(int fEntrada, int lectura, int escritura, char *buffer, int tamanyo){
53
54     // El padre solo escribe en este tubo, asi que cierra el extremo de lectura.
55     close(lectura);
56
57     int numLeidos = read(fEntrada, buffer, tamanyo);
58     write(escritura, buffer, numLeidos);
59
60     // Al cerrar el extremo de escritura el hijo sabe que no llegaran mas datos
61     // y su bucle de lectura termina.
62     close(escritura);
63 }

```

4.5. Función principal

Recibe los argumentos introducidos como parámetros, si los argumentos introducidos no son correctos se mostrará un mensaje de error por pantalla. Si son correctos se continuará con la ejecución de la función partir, la encargada de realizar la partición del archivo.

```

116 // ----- Funcion principal -----
117
118 int main(int argc, char *argv[]){
119
120     if(argc != 3){
121         printf("Error. Uso: %s fichero tam_trozo\n", argv[0]);
122     }
123     else{
124         int tamanyo = atoi(argv[2]);
125
126         // Si tamanyo fuera 0 o negativo, calcularNumTrozos dividiria entre 0.
127         if(tamanyo <= 0){
128             printf("Error. tam_trozo debe ser un entero positivo\n");
129         }
130         else{
131             partir(argv[1], tamanyo);
132         }
133     }
134     return 0;
135 }

```

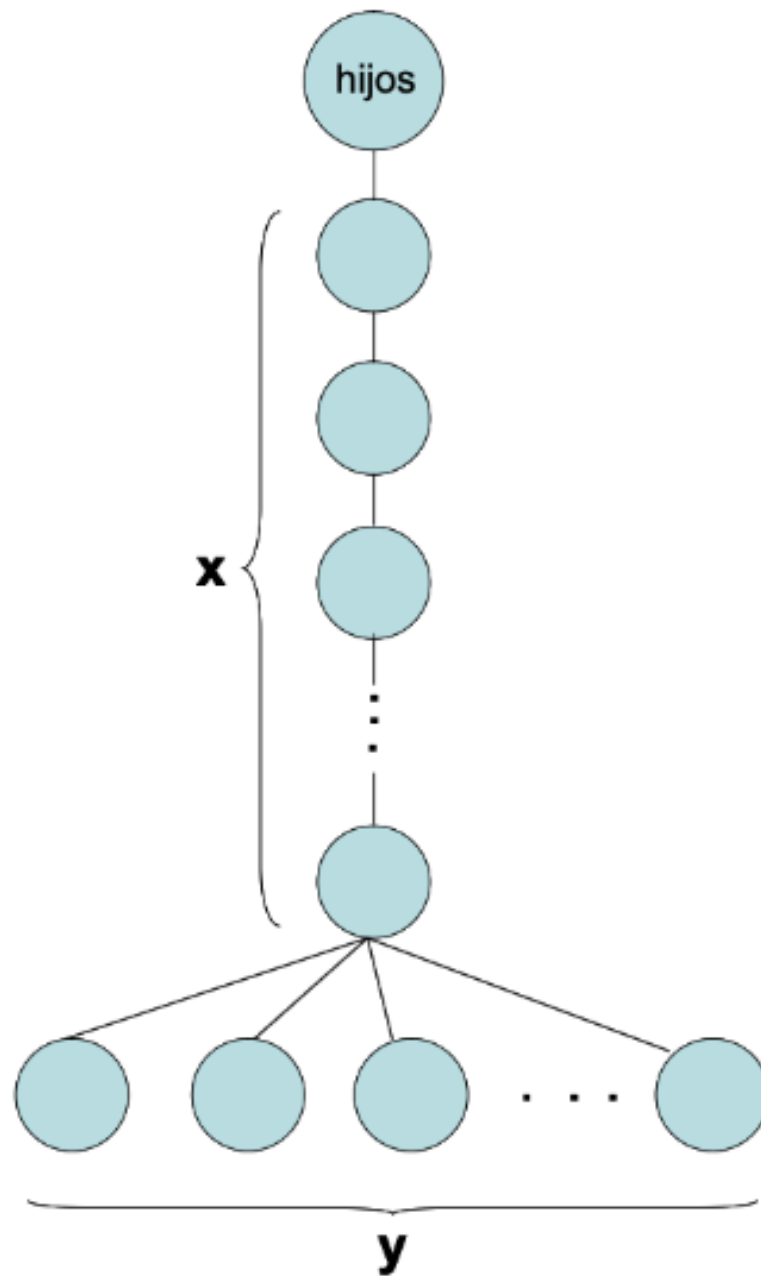
4. Ejercicio 3

1. Enunciado del ejercicio

Diseña un programa llamado **hijos.c** que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros. El proceso **hijos** debe mostrar el siguiente mensaje: “Soy el superpadre (pid) : mis hijos finales son: `pidx1, pidx2, pidx3, ..., pidxy`”

Los procesos de la parte baja deben mostrar el siguiente mensaje: “Soy el subhijo `pidx1`, mi padres son: `pid1, pid2, ..., pidx`”

1.1. Estructura del árbol de procesos



1.2. Resultado esperado

1.2.1. Ejemplo 1: 2x2

```
ubuntu@ubuntu:~/Documents/S0/P1/Ejercicio1/Solucion$ ./hijos 2 2
Proceso 4253 creado. Padre: 4252
Mostrando la estructura de procesos mediante pstree...
Proceso 4255 creado. Padre: 4252, 4253
hijos—hijos—hijos—hijos
      |             |
      +-----pstree
Proceso principal 4252. Hijos finales: 4256, 4257
```

1.2.2. Ejemplo 2: 3x4

```
ubuntu@ubuntu:~/Documents/SO/P1/Ejercicio1/Solucion$ ./hijos 3 4
Proceso 4266 creado. Padre: 4265
Mostrando la estructura de procesos mediante pstree...
Proceso 4268 creado. Padre: 4265, 4266
Proceso 4269 creado. Padre: 4265, 4266, 4268
hijos—hijos—hijos—hijos—hijos
      |             |             |
      |             |             |
      |             |             |
      |             |             |
pstree
Proceso principal 4265. Hijos finales: 4270, 4271, 4272, 4273
```

1.2.3. Ejemplo 3: 6x7

```
ubuntu@ubuntu: ~/Documents/SO/P1/Ejercicio1/Solucion$ ./hijos 6 7
Proceso 4275 creado. Padre: 4274
Mostrando la estructura de procesos mediante pstree...
Proceso 4277 creado. Padre: 4274, 4275
Proceso 4278 creado. Padre: 4274, 4275, 4277
Proceso 4279 creado. Padre: 4274, 4275, 4277, 4278
Proceso 4280 creado. Padre: 4274, 4275, 4277, 4278, 4279
Proceso 4281 creado. Padre: 4274, 4275, 4277, 4278, 4279, 4280
hijos—hijos—hijos—hijos—hijos—hijos—hijos—hijos
|
|_pstree
|
Proceso principal 4274. Hijos finales: 4282, 4283, 4284, 4285, 4286, 4287, 4288
```

2. Justificación e Identificación de Tareas

2.1. Recepción y validación de argumentos

El programa comienza con la llamada `./hijos <x> <y>`, siendo X e Y las dimensiones del árbol, estos parámetros los verificará la función `Parse()`, que se encarga de verificar el formato de los argumentos. Si los argumentos no son válidos, se mostrará un mensaje por pantalla, si son válidos, se continuará con la correcta ejecución del programa.

2.2. Creación de los segmentos de memoria compartida

Mediante el uso de las llamadas `shmget` y `shmat` reservamos la memoria necesaria para los PIDs de los hijos finales, y conectamos el proceso principal (el padre) con el segmento de memoria reservada respectivamente.

2.3. Creación del árbol de procesos

Posteriormente el proceso padre comienza a crear el árbol, primero con los hijos de la estructura vertical, dejándolos todos dormidos mientras el último hijo crea las ramas de los hijos finales.

2.4. Impresión de los PIDs de los hijos finales

De los hijos de dichas ramas extraemos su PID y lo guardamos en el vector de memoria reservada. Posteriormente el proceso padre utilizará ese mismo vector para imprimir por pantalla los PIDs de los procesos de sus hijos finales, justo después de imprimir por pantalla el árbol de procesos con PSTREE.

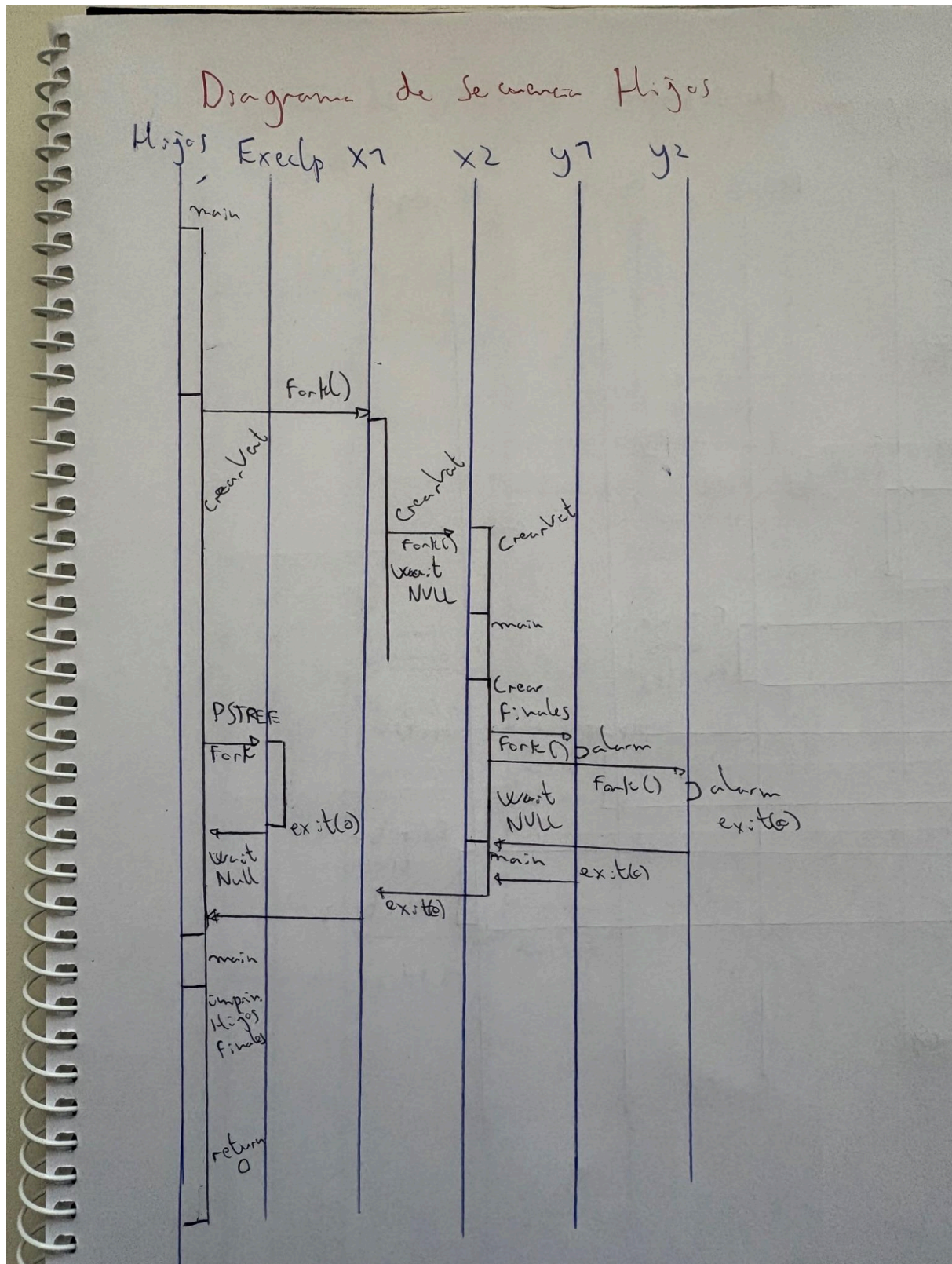
2.5. Desconexión y liberación de la memoria compartida

Después de imprimir los PIDs de los hijos finales, el proceso padre desde la función principal desconecta y libera la memoria compartida mediante el uso de las llamadas al sistema shmdt y shmctl.

2.6. Ejecución de la alarma y destrucción controlada del árbol

Después de la correcta creación del árbol y ejecución del PSTREE, el programa debe destruirse con una estricta jerarquía. Este orden comienza con la muerte de los hijos finales, mediante alarmas. Posteriormente irán despertando y muriendo sus progresivos padres, abuelos, bisabuelos, tatarabuelos, trastatarabuelos, y un casi infinito etc... Tras la muerte de toda la estructura vertical, el programa principal terminará la ejecución del programa con éxito.

3. Diagrama de Secuencia



4. Análisis detallado del Código fuente

4.1. Funciones globales y de utilidad

4.1.1. Función Parse()

Esta función se encarga de la gestión de argumentos y de comprobar si son correctos. Recibe mediante parámetros la cantidad de argumentos (int argc) y el array con cada uno de ellos (char* argv[]). En el caso de que no se haya introducido la cantidad correcta de argumentos, devolverá false. Igual que si alguno de los valores introducidos por terminal son negativos. Esto detendrá la ejecución del programa mostrando un mensaje de error por pantalla.

```
36 bool parse(int argc, char *argv[])
37 {
38     if (argc != 3) {
39         printf("Uso correcto: ./hijos <x columnas> <y filas>\n");
40         return false;
41     }
42     int x = atoi(argv[1]); /* numero de filas */
43     int y = atoi(argv[2]); /* numero de columnas */
44
45     if(x <= 0 || y <= 0){
46         printf("Error: los valores introducidos deben ser enteros positivos\n");
47         return false;
48     }
49     else{
50         return true;
51     }
52 }
```

4.1.2. Función EjecutarPstree()

Esta función recibe mediante parámetros el Pid del proceso padre, y ejecutará la llamada a PSTREE mediante la llamada al sistema execlp().

```

23 void ejecutarPstree(pid_t pidHijos){
24
25     printf("Mostrando la estructura de procesos mediante pstree...\n");
26
27     char pidt[20];
28     sprintf(pidt, "%d", pidHijos);
29     execlp("pstree", "pstree", "-c", pidt, NULL);
30     exit(1);
31
32 }

```

4.1.3. Función de Vacío()

Función de vacío que no recibe ningún parámetro y que únicamente sirve para despertar a los hijos finales de la alarma.

4.2. Funciones de construcción del árbol

4.2.1. Función CrearVertical()

Esta función comienza con un bucle for que irá creando cada uno de sus procesos hijos. En el caso de que el pid sea del hijo, guardará su PID en el vector de X e imprimirá por pantalla los PIDs del vector X guardados hasta ese momento, en cambio si el pid es de un padre, este hibernará mediante un wait(NULL) esperando la pronta muerte de su respectivo hijo, y si el PID es el del proceso principal Hijos, este ejecutará el PSTREE con todo el tronco y ramas del árbol de procesos. De la ejecución de esta función, al principio solo podrá salir el último hijo, que posteriormente ejecutará crearHijosFinales() para terminar de construir la estructura completa del árbol. Finalmente devolverá el valor de la variable func para

que la función principal verifique que se han creado todos los procesos.

```
56 int crearVertical(int func, int x, int y, int *vectorX, pid_t pidHijos){
57
58     for(func = 1; func <= x; func++){
59
60         pid_t pid = fork();
61
62         if (getpid() == pidHijos){
63
64             pid = fork();
65
66             if(pid == 0){
67                 ejecutarPstree(pidHijos);
68             }
69
70             wait(NULL);
71         }
72         if(pid != 0){
73
74             wait(NULL); // El proceso padre espera a que termine su hijo
75             break;      // Una vez terminado, deja de crear procesos verticales y sale del bucle
76         }
77         else{
78             // Aquí se almacena el identificador del nuevo proceso en el vector.
79             vectorX[func - 1] = getpid();
80             printf("Proceso %d creado. Padre: ", getpid());
81             printf("%d", pidHijos);
82             // Se muestran los procesos verticales creados previamente.
83
84             /*
85             En cada nueva posición se muestran los PIDs
86             de los procesos anteriores a la cadena
87             */
88
89             for(int j = 0; j < func - 1; j++){
90                 printf(", %d", vectorX[j]);
91             }
92             printf("\n");
93         }
94     }
95
96     return func;
97 }
```

4.2.2. Función CrearHijosFinales()

Esta función comienza creando cada uno de los hijos finales mediante un bucle for. Si se trata del hijo, este guarda su PID en el vector de memoria compartida de Y, configura la alarma y su llamada a la función de vacío y la ejecuta. Pasado el tiempo de espera de la alarma el proceso hijo muere con un exit0. Al final de

esta función tenemos otro bucle for al que solamente entra el proceso padre de todos esos hijos, y se encarga de realizar tantos wait(NULL) como hijos tenga. Tras la muerte de todos sus hijos el proceso padre seguirá con la ejecución de la función principal hasta su muerte.

```
114 void crearHijosFinales(int func, int y, int *vectorY){
115
116     pid_t pid;
117
118     for(func = 1; func <= y; func++){
119
120         pid = fork();
121
122         if(pid == 0){
123
124             vectorY[func - 1] = getpid(); // Guardamos el PID del proceso creado
125             signal(SIGALRM, vacio);
126             alarm(4);
127             pause();
128             exit(0); // El hijo termina su ejecucion con exito
129         }
130     }
131     if(func == y + 1){ // Solo el proceso padre llega hasta este punto
132
133         for(func = 1; func <= y; func++){
134
135             wait(NULL); // El padre espera a todos los hijos horizontales
136         }
137     }
138 }
```

4.2.3. Función ImprimirHijosFinales()

Esta función únicamente la ejecuta el super padre y se encarga de mostrar por pantalla todos y cada uno de los PIDs de los hijos finales accediendo al segmento de memoria compartida, donde se han guardado estos.

```

101 void imprimirHijosFinales(int func, int y, int *vectorY){
102
103     printf("Proceso principal %d. Hijos finales: ", getpid());
104
105     for(func = 0; func < y - 1; func++){
106
107         printf("%d, ", vectorY[func]); // Se imprimen todos salvo el ultimo
108     }
109     printf("%d\n", vectorY[y - 1]); // El ultimo PID se muestra por separado sin coma
110 }

```

4.3. Función principal

4.3.1. Función Main()

La función principal, que recibe como parámetros los argumentos, que gestionará la función `parse()`, en el caso de que los argumentos sean correctos, guardamos el valor de las medidas del árbol de procesos. Guardamos el PID del super padre para después pasárselo a la función `crearVertical()`. Y comenzamos la creación y conexión de los vectores segmento de memoria compartida. Tras la creación de la memoria, ejecutamos la función `crearVertical()`, que creará la estructura vertical de procesos. Posteriormente tenemos dos condicionales `if`, el primero es para comprobar si se trata del proceso padre principal, este constará de la variable 'func' con valor 1, porque en `crearVertical()` salió con anticipación del bucle, dentro de este condicional el proceso padre ejecutará la función `imprimirHijosFinales()` y finalmente desconectará y liberará los segmentos de memoria utilizados. El segundo `if` comprueba de si se trata del hijo final, que creará la estructura horizontal de procesos hijos finales. Tras mostrar los PIDs de los hijos finales y liberar la memoria usada, el programa terminará con éxito su ejecución.

```

142 int main(int argc, char *argv[])
143 {
144     if (!parse(argc, argv)) {
145         printf("Error: los argumentos introducidos no son validos.\n");
146         exit(1);
147     }
148     else{
149         int func, x, y, shmidx, shmidy;
150         int *vectorX, *vectorY; // Punteros utilizados para acceder a la memoria compartida
151         pid_t pidHijos;
152
153         x = atoi(argv[1]);
154         y = atoi(argv[2]);
155
156         pidHijos = getpid();
157
158         // Reservamos dos segmentos de memoria compartida para almacenar los PIDs.
159
160         // Este segmento contiene los procesos de la parte vertical.
161         shmidx = shmget(IPC_PRIVATE, sizeof(int) * x, IPC_CREAT | 0666);
162         vectorX = (int *) shmat (shmidx, 0, 0);
163
164         // Este otro segmento contiene los procesos de la parte horizontal.
165         shmidy = shmget(IPC_PRIVATE, sizeof(int) * y, IPC_CREAT | 0666);
166         vectorY = (int *) shmat (shmidy, 0, 0);
167
168         // Construimos la cadena vertical de procesos.
169         func = crearVertical(func, x, y, vectorX, pidHijos);
170
171         if(func == 1){ // Si func tiene valor 1, estamos en el padre principal, porque ha salido del bucle con anticipación
172             imprimirHijosFinales(func, y, vectorY);
173
174             // Desconectamos y liberamos la memoria los segmentos compartidos
175             shmdt(vectorX);
176             shmdt(vectorY);
177             shmctl(shmidx, IPC_RMID, NULL);
178             shmctl(shmidy, IPC_RMID, NULL);
179         }
180         elseif(func == x + 1){ // El ultimo proceso vertical se encarga de crear los procesos finales
181             crearHijosFinales(func, y, vectorY);
182         }
183     }
184
185     return 0;
186 }

```